MASTER 1 BIG DATA · UCAO-UUT

Taux de Rendement Synthétique et maintenance prédictive

Atelier de production — Zone Franche du Togo

Rapport technique : données, modèles, entraînement et évaluation

Auteur	KOUTSAVA Kossi Epiphane
Formation	Master 1 Big Data — UCAO-UUT
Unité d'enseignement	INF2224 — Extraction d'Information
Code source	`github.com/kkepiphane/trs-zone-franche-tg`
Date	Octobre 2026

À propos des données

Les données de production industrielles étant confidentielles, ce travail s'appuie sur un **jeu de données simulé de façon réaliste**, avec un scénario de dégradation contrôlé. Les résultats chiffrés illustrent et valident la démarche ; ils ne constituent pas une mesure de performance sur un atelier réel.

Résumé

Ce rapport décrit une chaîne de traitement de données complète, de la donnée brute de production jusqu'à la prédiction, appliquée au suivi d'un atelier de cinq machines. La performance est mesurée par le **Taux de Rendement Synthétique** (TRS), produit de la disponibilité, de la performance et de la qualité. Deux modèles d'apprentissage sont construits : une **régression linéaire** qui projette la tendance du TRS de chaque machine, et une **forêt aléatoire** (Random Forest) qui estime la probabilité qu'une défaillance survienne dans les cinq jours ouvrés suivants, à partir d'indicateurs calculés sur une fenêtre glissante.

Sur le jeu simulé (450 relevés journaliers, TRS global de 79,3 %), la machine M-03 est identifiée comme critique : tendance de $-0{,}31$ point de TRS par jour ($R^2 = 0{,}79$) et probabilité de panne estimée à 90 %. Sur le jeu de test, le classifieur obtient une exactitude de 98,3 % mais un rappel de 33 % au seuil de décision par défaut. L'analyse montre que l'exactitude est trompeuse sur des classes aussi déséquilibrées (2,25 % de cas positifs), que le classement des risques est en réalité parfait (AUC $= 1$) et qu'un seuil de 0,2 permet de détecter toutes les pannes du test au prix d'une fausse alerte. Les limites liées à la taille de l'échantillon et au découpage des données sont discutées. L'ensemble est livré dans un tableau de bord interactif (Plotly Dash).

Mots-clés : TRS, OEE, maintenance prédictive, Random Forest, classes déséquilibrées, régression linéaire, tableau de bord.

Table des matières

Résumé **1**

1 Introduction **3**
1.1 Contexte . . . 3
1.2 Objectifs . . . 3
1.3 Démarche . . . 3

2 Données **3**
2.1 Origine : un jeu simulé au scénario contrôlé . . . 3
2.2 Schéma des données . . . 4
2.3 Préparation et contrôle qualité . . . 4

3 Le Taux de Rendement Synthétique **4**
3.1 Définition . . . 4
3.2 Résultats . . . 5
3.3 Diagnostic . . . 5

4 Analyse exploratoire **5**

5 Modèle 1 : prévision de la tendance du TRS **6**
5.1 Formulation . . . 6
5.2 Résultats . . . 7

6 Modèle 2 : prédiction du risque de panne **7**
6.1 Formulation du problème . . . 7
6.2 Construction des variables explicatives . . . 8
6.3 Jeu d'apprentissage . . . 8
6.4 Séparation apprentissage / test . . . 8
6.5 Modèle : forêt aléatoire . . . 8
6.6 Évaluation . . . 9
6.6.1 Métriques . . . 9
6.6.2 Résultats au seuil par défaut . . . 9
6.6.3 Le seuil de décision, pas le classement . . . 10
6.6.4 Stabilité : validation croisée . . . 10
6.7 Interprétation : importance des variables . . . 10
6.8 Score de risque actuel . . . 11

7 Mise en œuvre logicielle **11**
7.1 Architecture . . . 11
7.2 Tableau de bord . . . 11

8 Limites et perspectives **12**

9 Conclusion **13**

A Reproduire les résultats **14**

1 Introduction

1.1 Contexte

Dans les entreprises industrielles de la Zone Franche du Togo, comme dans toute industrie manufacturière, un arrêt machine non planifié se traduit par de la production perdue, des retards de livraison et des coûts d'intervention en urgence. Trois stratégies de maintenance coexistent :

— la maintenance **corrective**, qui intervient après la panne ;

— la maintenance **préventive**, qui intervient à intervalles fixes, que la machine en ait besoin ou non ;

— la maintenance **prédictive**, qui intervient lorsque les données indiquent qu'une défaillance devient probable.

La maintenance prédictive suppose de disposer de données de production, d'un indicateur de performance fiable et de modèles capables d'anticiper une dégradation. C'est l'objet de ce projet.

1.2 Objectifs

1. **Mesurer** la performance de chaque machine par le TRS et ses trois composantes.
2. **Diagnostiquer** la composante qui pénalise le plus chaque machine.
3. **Prévoir** l'évolution du TRS à court terme.
4. **Prédire** le risque de défaillance de chaque machine dans les jours qui suivent.
5. **Restituer** ces résultats dans un outil d'aide à la décision utilisable par un responsable de production.

1.3 Démarche

La figure 1 résume la chaîne de traitement. L'exploration et l'entraînement sont réalisés dans un notebook Jupyter ; le modèle obtenu est exporté puis chargé par le tableau de bord, selon la séparation usuelle entre expérimentation et mise en production.

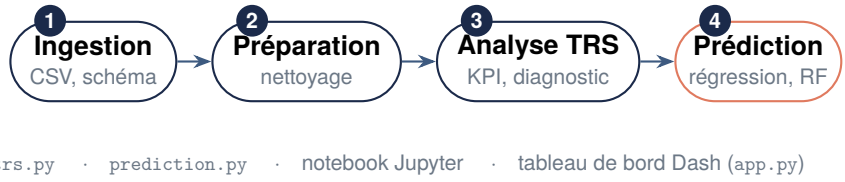


Figure 1. Chaîne de traitement, de la donnée brute à la prédiction.

2 Données

2.1 Origine : un jeu simulé au scénario contrôlé

Les données sont produites par le script `generer_donnees.py` (graine aléatoire fixée à 42, donc reproductibles). Elles décrivent un atelier de **5 machines** (M-01 à M-05) suivies pendant **90 jours ouvrés**, du 02/02/2026 au 05/06/2026, soit **450 relevés journaliers**.

Chaque machine possède un profil de base (disponibilité, performance, qualité) autour duquel les valeurs journalières varient selon une loi normale. Une **dérive** de la disponibilité est ajoutée : nulle pour M-01, M-02 et M-05, modérée pour M-04 ($-0{,}0015$ par jour) et forte pour **M-03** ($-0{,}0032$ par jour). Les signaux d'usure (arrêts, micro-arrêts, température moteur) sont

tirés à partir de la perte de disponibilité, ce qui reproduit le comportement attendu d'une machine qui se dégrade.

> **Pourquoi un scénario contrôlé ?**
> Connaître à l'avance la machine qui se dégrade permet de **vérifier** que la chaîne détecte bien une anomalie connue, avant de l'appliquer à des données réelles. En contrepartie, la relation entre signaux d'usure et disponibilité est construite par le générateur : le modèle la retrouve plus facilement qu'il ne le ferait sur des données d'usine (voir section 8).

2.2 Schéma des données

Le tableau 1 décrit les colonnes. Les sept premières sont obligatoires pour calculer le TRS ; les trois dernières alimentent le modèle de risque de panne.

Table 1. Schéma du fichier de production (un relevé par machine et par jour).

Colonne	Unité	Description
`machine`	—	identifiant de la machine
`date`	—	jour de production
`temps_prevu`	min	temps de production planifié (T_p, 450 min par jour)
`temps_marche`	min	temps de fonctionnement effectif (T_m)
`prod_theorique`	pièces	production atteignable à la cadence nominale pendant T_m (N_{th})
`prod_reelle`	pièces	pièces effectivement produites (N_r)
`pieces_bonnes`	pièces	pièces conformes (N_b)
`nb_arrets`	nombre	arrêts de la journée
`micro_arrets`	nombre	arrêts de courte durée
`temp_moteur`	°C	température moyenne du moteur

2.3 Préparation et contrôle qualité

Avant tout calcul, la fonction `nettoyer()` applique les règles suivantes :

1. suppression des doublons exacts ;
2. suppression des lignes dont une valeur obligatoire est manquante ;
3. cohérence physique : $T_m \le T_p$ et $N_b \le N_r$;
4. dénominateurs strictement positifs : $T_p > 0$, $N_{th} > 0$, $N_r > 0$.

Sur le jeu simulé, aucune ligne n'est rejetée (450 sur 450). Ces règles sont néanmoins indispensables sur des données réelles, saisies manuellement ou issues de capteurs. Le tableau de bord affiche le nombre de lignes rejetées et les valeurs manquantes à chaque import.

3 Le Taux de Rendement Synthétique

3.1 Définition

Pour une machine et une journée donnée, les trois composantes et le TRS sont définis par :

$$D = \frac{T_m}{T_p}, \qquad P = \frac{N_r}{N_{th}}, \qquad Q = \frac{N_b}{N_r}, \qquad \mathrm{TRS} = D \times P \times Q. \tag{1}$$

La **disponibilité** D mesure les pertes dues aux arrêts, la **performance** P les pertes de cadence (micro-arrêts, ralentissements) et la **qualité** Q les pertes dues aux rebuts. Un TRS de 85 % est communément retenu comme référence de « classe mondiale » ; c'est l'objectif fixé dans ce projet.

Le TRS d'une machine sur une période est la moyenne de ses TRS journaliers ; le TRS de l'atelier est la moyenne sur l'ensemble des relevés. Comme la moyenne d'un produit n'est pas le produit des moyennes, ce TRS global diffère légèrement du produit des trois composantes moyennes (moins de 0,1 point ici).

3.2 Résultats

Le TRS de l'atelier s'établit à **79,3 %**, soit 5,7 points sous l'objectif. Trois machines sur cinq sont sous l'objectif (tableau 2, figure 2).

Table 2. Composantes moyennes et TRS par machine, du plus faible au plus élevé.

Machine	Disponibilité	Performance	Qualité	TRS
M-03	71,4 %	94,9 %	98,1 %	66,5 %
M-04	83,7 %	91,7 %	95,8 %	73,5 %
M-02	89,4 %	93,0 %	97,2 %	80,8 %
M-01	93,2 %	94,3 %	99,0 %	86,9 %
M-05	93,8 %	95,7 %	98,8 %	88,7 %
Atelier	86,3 %	93,9 %	97,8 %	**79,3 %**

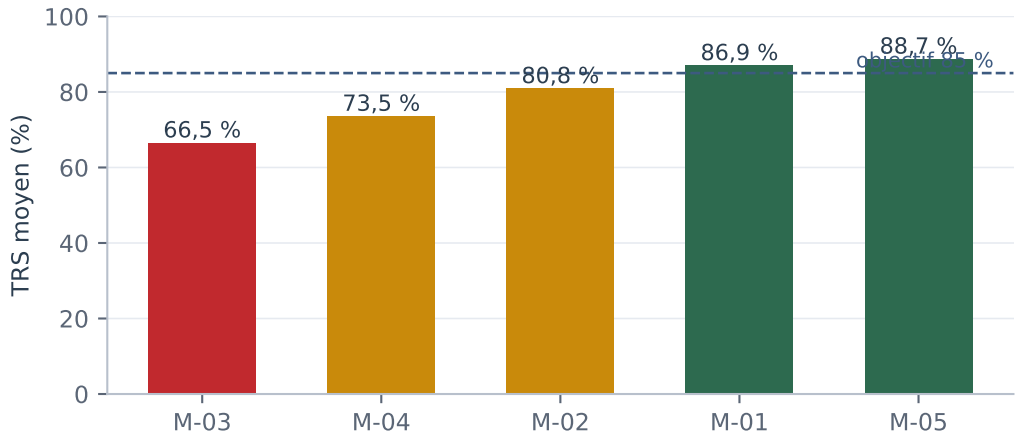


Figure 2. TRS moyen par machine (vert : objectif atteint ; ocre : entre 70 et 85 % ; rouge : sous 70 %).

3.3 Diagnostic

Pour chaque machine, la composante la plus faible indique où agir en priorité. Pour M-03, c'est nettement la **disponibilité** (71,4 %), alors que sa performance et sa qualité sont comparables à celles des autres machines : la machine *s'arrête* de plus en plus. L'évolution hebdomadaire (figure 3) confirme une baisse continue, d'environ 80 % en février à environ 53 % début juin. M-04 suit la même tendance, de façon plus modérée.

4 Analyse exploratoire

La matrice de corrélation de Pearson (figure 4) entre les relevés journaliers fait apparaître deux constats :

— le TRS est porté essentiellement par la **disponibilité** ($r = 0{,}94$), bien plus que par la performance ($r = 0{,}38$) ou la qualité ($r = 0{,}33$) ;

— les **arrêts** ($r = -0{,}69$), **micro-arrêts** ($r = -0{,}71$) et la **température moteur** ($r = -0{,}73$)

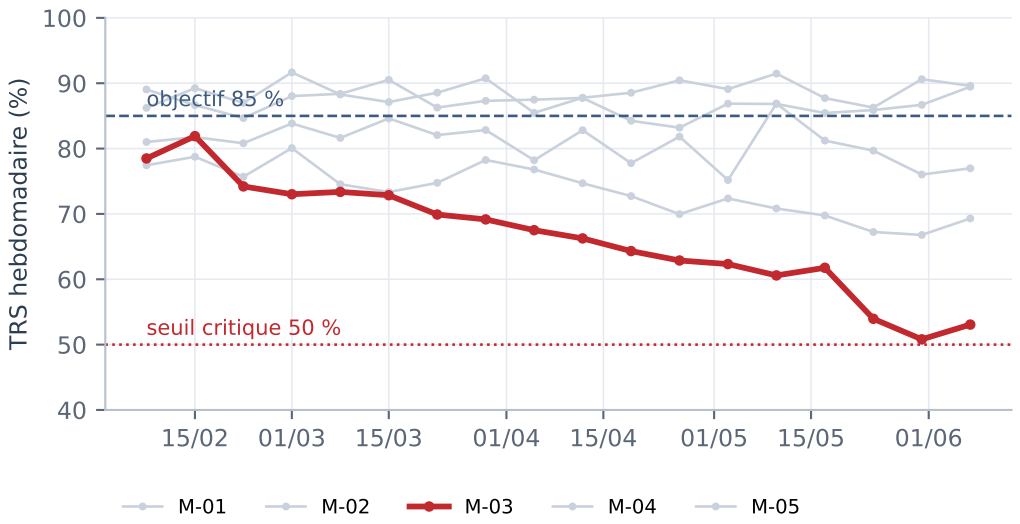


Figure 3. Évolution hebdomadaire du TRS. M-03 (en rouge) se rapproche du seuil critique de 50 %.

sont fortement et négativement corrélés au TRS : ce sont des *signaux précurseurs* candidats pour la prédiction de panne.

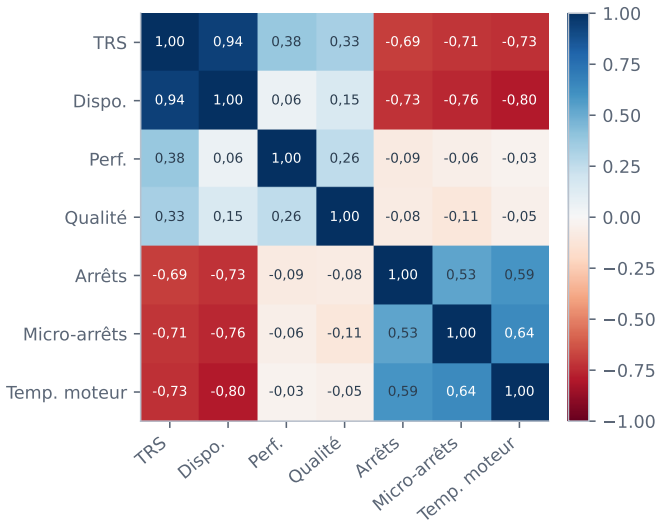


Figure 4. Corrélations entre composantes du TRS et signaux d'usure (relevés journaliers).

5 Modèle 1 : prévision de la tendance du TRS

5.1 Formulation

Pour chaque machine, le TRS journalier y_t est modélisé comme une fonction affine du rang t du jour ouvré ($t = 0, \dots, n-1$) :

$$y_t = \beta_0 + \beta_1 \, t + \varepsilon_t. \tag{2}$$

Les coefficients sont estimés par moindres carrés ordinaires (`LinearRegression` de scikit-learn), c'est-à-dire en minimisant $\sum_t (y_t - \beta_0 - \beta_1 t)^2$, ce qui donne :

$$\hat{\beta}_1 = \frac{\sum_t (t - \bar{t})(y_t - \bar{y})}{\sum_t (t - \bar{t})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{t}. \tag{3}$$

La pente $\hat{\beta}_1$ s'interprète directement comme la **variation du TRS par jour ouvré**. La projection à l'horizon h est $\hat{y}_{n-1+h} = \hat{\beta}_0 + \hat{\beta}_1(n-1+h)$, bornée à $[0,1]$, et datée sur les jours ouvrés suivants. La qualité de l'ajustement est mesurée par le coefficient de détermination $R^2 = 1 - \sum_t (y_t - \hat{y}_t)^2 / \sum_t (y_t - \bar{y})^2$.

5.2 Résultats

Table 3. Tendance du TRS par machine (90 jours ouvrés).

Machine	Pente (pt / jour)	R^2	Lecture
M-03	**−0,314**	**0,791**	dégradation nette et régulière
M-04	−0,129	0,324	dégradation modérée
M-02	−0,046	0,058	stable (pas de tendance significative)
M-01	−0,016	0,007	stable
M-05	+0,014	0,006	stable

Pour M-03, la droite explique 79 % de la variance du TRS : la tendance est à la fois forte et régulière. Projetée sur 15 jours ouvrés, elle conduit à un TRS de **47,8 %** et franchit le seuil critique de 50 % au **9e jour ouvré** (figure 5). Pour les autres machines, le R^2 faible indique que les variations sont dominées par le bruit journalier : aucune dérive n'est à signaler.

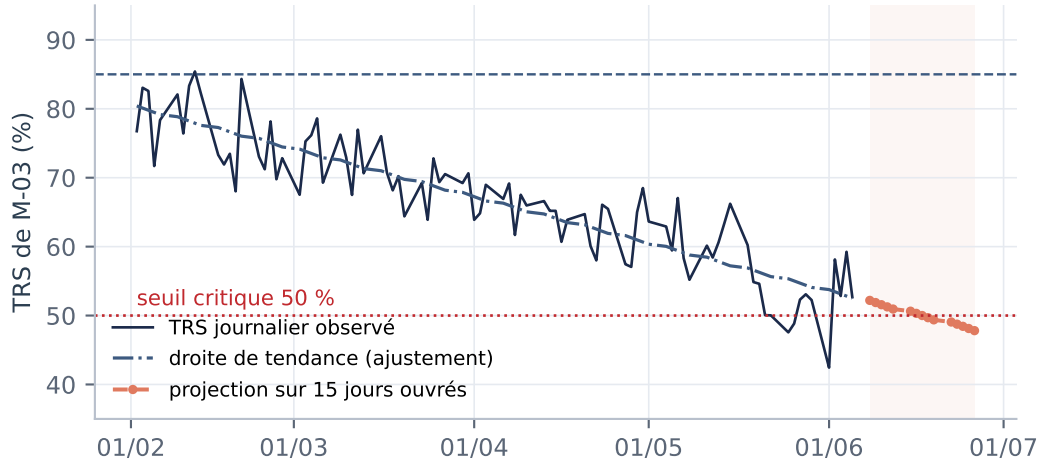


Figure 5. TRS journalier de M-03, droite de tendance ajustée et projection sur 15 jours ouvrés.

> **Intérêt opérationnel.** La pente et la date de franchissement du seuil donnent au responsable de maintenance un *délai d'action* : ici, environ deux semaines pour planifier une intervention sur M-03 avant que son TRS ne devienne critique.

6 Modèle 2 : prédiction du risque de panne

6.1 Formulation du problème

On définit une **défaillance** comme une journée où le TRS d'une machine passe sous le seuil critique $s = 0{,}50$. Le problème est posé comme une **classification binaire supervisée** : à la date t, pour la machine m, on cherche à prédire si une défaillance surviendra dans les $H = 5$ jours

ouvrés suivants :

$$y_{m,t} = \begin{cases} 1 & \text{s'il existe } j \in \{t, \dots, t+H-1\} \text{ tel que } \mathrm{TRS}_{m,j} < s, \\ 0 & \text{sinon.} \end{cases} \tag{4}$$

6.2 Construction des variables explicatives

L'état récent de la machine est résumé sur une **fenêtre glissante** des $W = 5$ jours ouvrés *précédant* t, notée $\mathcal{F}_t = \{t-W, \dots, t-1\}$. Six variables sont calculées (tableau 4).

Table 4. Variables explicatives, calculées sur la fenêtre $\mathcal{F}_t$ des 5 jours précédents.

Variable	Calcul	Ce qu'elle capte
`trs_moyen`	moyenne du TRS sur $\mathcal{F}_t$	niveau de performance récent
`trs_pente`	pente de la droite ajustée au TRS sur $\mathcal{F}_t$	dégradation en cours
`dispo_moyenne`	moyenne de D sur $\mathcal{F}_t$	pertes par arrêts
`arrets_moyens`	moyenne de `nb_arrets`	fréquence des arrêts
`micro_moyens`	moyenne de `micro_arrets`	instabilité de fonctionnement
`temp_moyenne`	moyenne de `temp_moteur`	échauffement, usure mécanique

Les variables n'utilisent que des jours *strictement antérieurs* à t, alors que la cible porte sur les jours t et suivants : aucune information future n'entre dans les variables.

6.3 Jeu d'apprentissage

Chaque machine fournit $90 - W - H = 80$ échantillons, soit **400 échantillons** au total. Seuls **9 sont positifs** (2,25 %), tous issus de M-03 : ils correspondent aux fenêtres qui précèdent les 3 journées où son TRS est passé sous 50 %. Les classes sont donc **fortement déséquilibrées**, ce qui conditionne tout le reste de la démarche.

6.4 Séparation apprentissage / test

Les échantillons sont répartis aléatoirement en 70 % pour l'apprentissage et 30 % pour le test (`train_test_split`, `random_state=42`). La séparation est **stratifiée** sur la cible, afin que la proportion de pannes soit la même dans les deux ensembles ; sans stratification, le jeu de test pourrait ne contenir aucune panne.

Table 5. Répartition des échantillons.

	Échantillons	Pannes	Taux de pannes
Apprentissage	280	6	2,1 %
Test	120	3	2,5 %

6.5 Modèle : forêt aléatoire

Principe. Une forêt aléatoire combine B arbres de décision. Chaque arbre est appris sur un échantillon *bootstrap* (tirage avec remise) du jeu d'apprentissage et, à chaque nœud, ne considère qu'un sous-ensemble aléatoire de variables ($\sqrt{6} \approx 2$ ici). Un nœud est découpé de façon à réduire au maximum l'**impureté de Gini** :

$$G = 1 - \sum_{c \in \{0,1\}} p_c^2, \tag{5}$$

où p_c est la proportion (pondérée) de la classe c dans le nœud. La probabilité de panne prédite est la moyenne des probabilités données par les B arbres : $\hat{p}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{p}_b(x)$. La décision est $\hat{y} = 1$ si $\hat{p}(x) \geq \tau$, avec un seuil $\tau = 0{,}5$ par défaut.

Ce modèle a été retenu pour sa robustesse sur de petits jeux de données tabulaires, l'absence de mise à l'échelle nécessaire, sa capacité à capter des interactions non linéaires et la mesure d'importance des variables qu'il fournit.

Pondération des classes. Avec `class_weight="balanced"`, chaque classe reçoit un poids inversement proportionnel à sa fréquence dans le jeu d'apprentissage :

$$w_c = \frac{n}{K\, n_c} \quad \Rightarrow \quad w_0 = \frac{280}{2 \times 274} \approx 0{,}51, \qquad w_1 = \frac{280}{2 \times 6} \approx 23{,}3. \tag{6}$$

Une erreur sur une panne coûte ainsi environ 46 fois plus qu'une erreur sur un cas sain lors de la construction des arbres, ce qui compense en partie le déséquilibre.

Table 6. Hyperparamètres du modèle (`RandomForestClassifier`, scikit-learn).

Paramètre	Valeur	Justification
`n_estimators`	200	assez d'arbres pour stabiliser la moyenne des votes
`max_depth`	6	limite la profondeur pour réduire le surapprentissage sur 280 échantillons
`class_weight`	`"balanced"`	compense le déséquilibre des classes
`max_features`	`"sqrt"` (défaut)	décorrèle les arbres
`criterion`	`"gini"` (défaut)	critère de découpage
`random_state`	42	reproductibilité

6.6 Évaluation

6.6.1 Métriques

Sur le jeu de test, on compte les vrais positifs (VP, pannes détectées), faux positifs (FP, fausses alertes), faux négatifs (FN, pannes manquées) et vrais négatifs (VN). On en déduit :

$$\text{Exactitude} = \frac{VP + VN}{VP + VN + FP + FN}, \qquad \text{Précision} = \frac{VP}{VP + FP}, \tag{7}$$

$$\text{Rappel} = \frac{VP}{VP + FN}, \qquad F_1 = 2\,\frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}. \tag{8}$$

En maintenance, une panne manquée (arrêt de production) coûte en général bien plus qu'une fausse alerte (une inspection inutile). Le **rappel** est donc la métrique prioritaire, la précision servant à vérifier que les alertes restent crédibles.

6.6.2 Résultats au seuil par défaut

Table 7. Performance sur le jeu de test (120 échantillons, dont 3 pannes), seuil $\tau = 0{,}5$.

Modèle	Exactitude	Précision	Rappel	F_1
Référence naïve : « toujours sain »	97,5 %	—	0 %	0
Forêt aléatoire, $\tau = 0{,}5$	98,3 %	100 %	**33,3 %**	0,50

Le modèle ne produit aucune fausse alerte, mais il ne détecte qu'**une panne sur trois** ($VP = 1, FN = 2, FP = 0, VN = 117$). L'exactitude de 98,3 % est trompeuse : un modèle qui répondrait toujours « sain » atteindrait déjà 97,5 % sans jamais détecter de panne. Sur des classes aussi déséquilibrées, l'exactitude n'est pas un critère pertinent.

On note aussi un surapprentissage : le modèle classe parfaitement le jeu d'apprentissage (exactitude et rappel de 100 %), ce qui est attendu avec si peu de cas positifs.

6.6.3 Le seuil de décision, pas le classement

Les probabilités prédites pour les trois pannes du test sont 0,93, 0,39 et 0,23. Deux d'entre elles sont sous le seuil de 0,5, mais toutes restent supérieures à celles de l'ensemble des cas sains : l'aire sous la courbe ROC vaut **AUC** = 1,00, de même que la précision moyenne (AP). Autrement dit, le modèle *classe* parfaitement les situations par niveau de risque ; c'est le *seuil* de 0,5 qui est inadapté à des probabilités tirées vers le bas par la rareté des pannes.

Table 8. Effet du seuil de décision τ sur le jeu de test.

τ	VP	FN	FP	**Précision**	**Rappel**	F_1
0,5	1	2	0	100 %	33 %	0,50
0,3	2	1	0	100 %	67 %	0,80
0,2	**3**	**0**	**1**	**75 %**	**100 %**	**0,86**

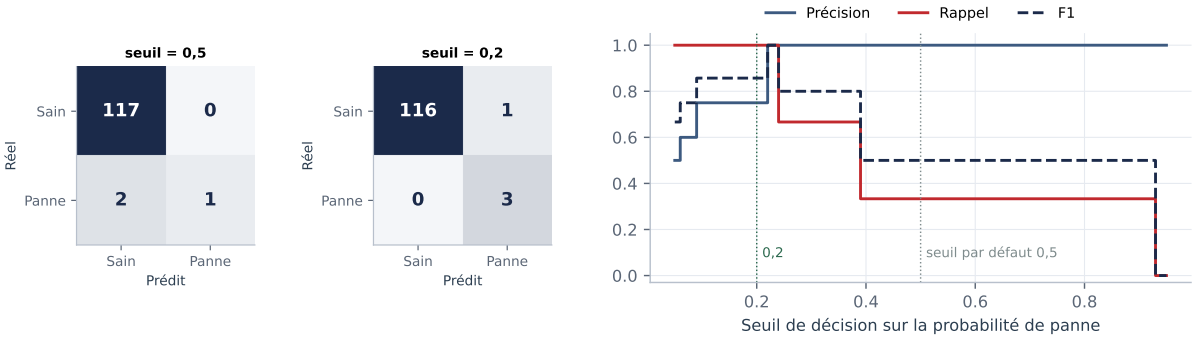


Figure 6. À gauche : matrices de confusion du jeu de test pour $\tau = 0{,}5$ et $\tau = 0{,}2$. À droite : précision, rappel et F_1 en fonction du seuil.

Avec $\tau = 0{,}2$, **les trois pannes sont détectées** au prix d'une seule fausse alerte sur 117 cas sains. Ce choix est cohérent avec la priorité donnée au rappel.

> **Précaution méthodologique**
>
> Le seuil ci-dessus est choisi en observant le jeu de test : présenté tel quel, ce résultat serait optimiste. Dans un cadre rigoureux, le seuil doit être fixé sur un jeu de *validation* distinct (ou par validation croisée), puis évalué une seule fois sur le test. Il est donné ici pour illustrer le levier, pas comme performance finale.

6.6.4 Stabilité : validation croisée

Avec seulement 3 pannes dans le test, chaque panne représente 33 points de rappel : la mesure est très instable. Une **validation croisée stratifiée à 5 plis** sur les 400 échantillons (chaque échantillon est prédit par un modèle qui ne l'a pas vu) donne, au seuil de 0,5, un rappel de **67 %** (6 pannes sur 9), une précision de 100 %, un F_1 de 0,80 et une AUC de 0,998. Le rappel de 33 % obtenu sur le découpage unique est donc plutôt pessimiste, mais l'écart entre les deux montre surtout qu'il faudrait davantage d'événements de panne pour conclure.

6.7 Interprétation : importance des variables

L'importance de chaque variable est mesurée par la diminution moyenne de l'impureté de Gini qu'elle apporte dans l'ensemble des arbres (figure 7).

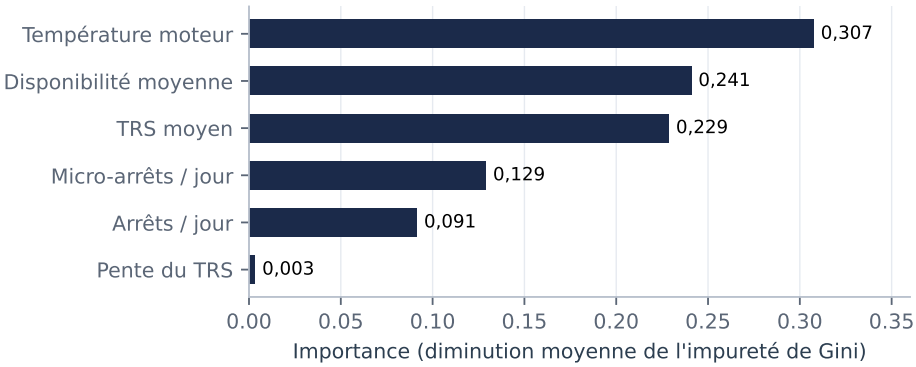


Figure 7. Importance des variables du modèle de risque.

La **température moteur** (0,307), la **disponibilité moyenne** (0,241) et le **TRS moyen** (0,229) dominent, suivis des micro-arrêts et des arrêts. Ce classement est cohérent avec l'analyse des corrélations et avec l'intuition métier : une machine qui chauffe et s'arrête souvent est proche de la panne. La pente sur 5 jours n'apporte presque rien (0,003) : calculée sur une fenêtre aussi courte, elle est dominée par le bruit journalier, alors que les niveaux moyens sont plus stables.

Deux réserves s'appliquent : l'importance de Gini favorise les variables continues et se répartit entre variables corrélées ; et, sur ce jeu simulé, la température est générée à partir de la perte de disponibilité, ce qui explique en partie son rang.

6.8 Score de risque actuel

Pour chaque machine, le modèle est appliqué à la fenêtre la plus récente et la probabilité $\hat{p}$ est traduite en niveau de risque : **faible** si $\hat{p} < 0{,}30$, **modéré** si $0{,}30 \leq \hat{p} < 0{,}60$, **élevé** si $\hat{p} \geq 0{,}60$.

Table 9. Risque de panne actuel par machine.

Machine	Probabilité de panne	Niveau
M-03	**0,905**	élevé
M-01, M-02, M-04, M-05	0,000	faible

M-03 est la seule machine signalée, ce qui est cohérent avec la prévision de tendance de la section précédente : les deux modèles convergent. Les probabilités nulles des autres machines traduisent un modèle très tranché, conséquence du faible nombre de cas positifs ; sur données réelles, une calibration des probabilités serait nécessaire.

7 Mise en œuvre logicielle

7.1 Architecture

Le code sépare la logique métier de l'interface, ce qui permet de la réutiliser telle quelle dans le notebook et dans l'application.

Le modèle exporté par le notebook est un dictionnaire `joblib` contenant le modèle entraîné, la liste des variables, la taille de fenêtre, l'horizon et le seuil de panne. Le tableau de bord le charge au démarrage ; s'il est absent, le modèle est entraîné à la volée avec les mêmes paramètres.

7.2 Tableau de bord

L'application Dash comporte cinq pages : vue d'ensemble, ingestion (import d'un CSV avec validation du schéma), préparation (rapport de qualité), analyse du TRS (graphiques liés : sé-

Table 10. Organisation du code (Python 3.10+).

Fichier	Rôle
`calcul_trs.py`	nettoyage, calcul de D, P, Q et du TRS, synthèse par machine
`prediction.py`	prévision par régression, construction des variables, entraînement, évaluation et score de risque
`maintenance_predictive_TRS.ipynb`	exploration, entraînement, évaluation ; export du modèle dans `modele_risque.joblib`
`app.py`	tableau de bord Plotly Dash (5 pages)
`generer_donnees.py`	génération du jeu simulé

lectionner une machine la met en avant dans tous les graphiques) et prédiction (risque par machine, prévision, métriques). La vue d'ensemble déclenche une alerte lorsqu'une machine présente un risque modéré ou élevé (figure 8).



Figure 8. Vue d'ensemble du tableau de bord : TRS décomposé, indicateurs et alerte de maintenance prédictive sur M-03.

8 Limites et perspectives

Données simulées. Les relations entre signaux d'usure et disponibilité sont définies par le générateur ; le modèle les retrouve donc plus facilement que sur des données réelles, bruitées et incomplètes. La principale perspective est l'application à des données d'usine, que l'outil accepte déjà (import CSV avec validation du schéma).

Très peu d'événements. Les 9 cas positifs proviennent de 3 journées de panne d'une seule machine, et les fenêtres glissantes qui se chevauchent sont fortement corrélées entre elles. Les métriques reposent donc sur très peu d'événements indépendants et doivent être lues comme indicatives.

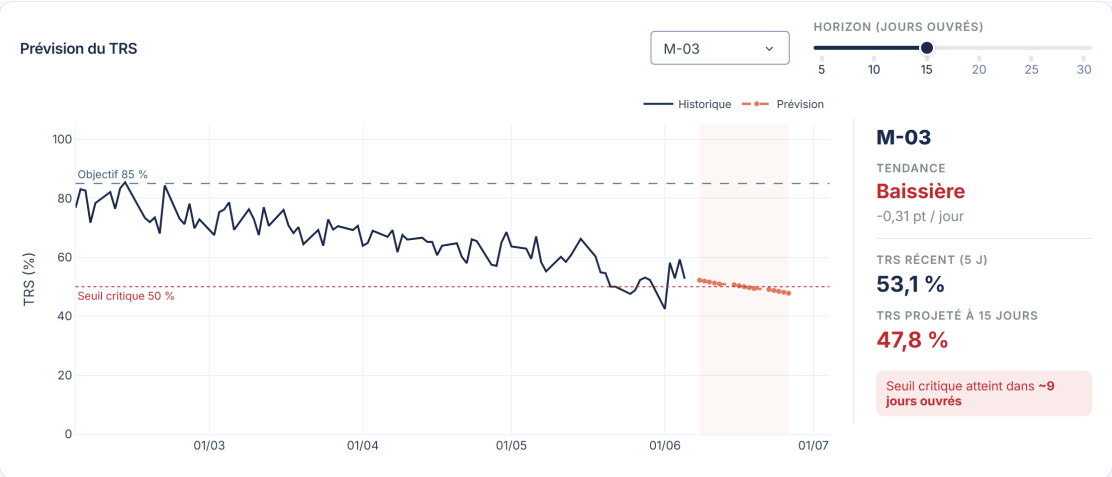


FIGURE 9. Page Prédiction : prévision du TRS de M-03 et délai estimé avant le seuil critique.

Découpage aléatoire de séries temporelles. Des fenêtres voisines dans le temps peuvent se retrouver l'une dans l'apprentissage, l'autre dans le test : l'évaluation est alors optimiste. Un découpage chronologique (apprendre sur le passé, tester sur le futur) serait plus rigoureux, mais il est impossible ici : toutes les pannes surviennent après le 28/04/2026, si bien qu'un découpage 70 %/30 % dans le temps laisse l'apprentissage sans aucune panne. Sur données réelles, on utilisera une validation temporelle (`TimeSeriesSplit`) et une validation par machine (`GroupKFold`) pour vérifier que le modèle se généralise à une machine non vue.

Pistes d'amélioration.

- choisir le seuil de décision en fonction des coûts réels (coût d'une panne contre coût d'une intervention), sur un jeu de validation ;
- rééquilibrer les classes (sur-échantillonnage SMOTE, sous-échantillonnage) et calibrer les probabilités ;
- enrichir les signaux (vibrations, consommation électrique, historique d'interventions) ;
- remplacer la tendance linéaire par des modèles de séries temporelles (ARIMA, lissage exponentiel) fournissant des intervalles de confiance ;
- comparer la forêt aléatoire à d'autres modèles (régression logistique, gradient boosting).

9 Conclusion

Ce projet met en place une chaîne complète, de la donnée brute de production à la prédiction, et la restitue dans un tableau de bord utilisable par un responsable de production. Sur le scénario simulé, les deux modèles convergent vers la même conclusion : la machine M-03, pénalisée par sa disponibilité, se dégrade d'environ 0,3 point de TRS par jour et présente un risque de panne élevé, ce qui laisse environ deux semaines pour planifier une intervention.

L'évaluation montre aussi les limites d'un apprentissage sur des événements rares : l'exactitude est trompeuse, le seuil de décision doit être adapté au coût d'une panne manquée, et la fiabilité des métriques dépend du nombre de pannes observées. Ces constats fixent la suite du travail : confronter l'outil à des données réelles et en valider les performances dans le temps.

A Reproduire les résultats

```
git clone https://github.com/kkepiphane/trs-zone-franche-tg.git
cd trs-zone-franche-tg
pip install -r requirements.txt

python generer_donnees.py                                  # données simulées
jupyter notebook maintenance_predictive_TRS.ipynb          # entraînement, export
python app.py                                              # tableau de bord : http ://127.0.0.1 :8050
python docs/rapport/generer_figures.py                     # figures de ce rapport
```

Toutes les sources d'aléa sont fixées : la graine `42` est utilisée pour la génération des données, pour la séparation apprentissage / test et pour la forêt aléatoire. Les chiffres de ce rapport sont donc reproductibles à l'identique.